Modelo Preguntas Parcial

API

- 1. ¿Qué entiende por **API?**
- 2. ¿Qué entiende por **API REST?**
- 3. ¿Por qué se asemeja una **API** al ejemplo del cliente que ordena al mozo una comida compleja?
- 4. ¿Qué diferencia hay entre una **API REST** y una **API?**
- 5. Dé un ejemplo de una **API** que no sea web.
- 6. ¿Qué relación existe entre una **API REST** y el token que devuelve el backend cuando un usuario se loguea?
- 7. ¿Qué diferencia hay entre una **API** y un endpoint? Dé un ejemplo.
- 8. ¿Qué significa que las **API REST** sean stateless? Dé un ejemplo. ¿Qué relación tiene con el token?
- 9. Explique cómo es el flujo de login de un usuario en el código de su Trabajo Práctico Obligatorio (TPO). Muestre el código fuente de su TPO.
- 11 Si, por ejemplo, me logueo en Mercado Libre y el backend no guarda ninguna información del usuario logueado, ¿cómo es que después puede realizar una compra a su nombre si no guarda ninguna información del usuario?
- 12. ¿Por qué en una **API REST** la característica de stateless permite la escalabilidad horizontal?
- 12.1 Dé un ejemplo de la situación cuando Mercado Libre promociona un Cyber Monday y las compras pasan de ser 10.000 por día a 100.000 por día.
- 13. ¿Por qué una **API REST** es más ligera? Compárela con SOAP.
- 14. Si estuviese haciendo un marketplace como Mercado Libre, ¿qué tipo de autenticación utilizaría y por qué?
- 15. Si estuviese haciendo un homebanking como Banco Ciudad, ¿qué tipo de arquitectura de comunicación utilizaría y por qué?
- 17. ¿Por qué el uso de **JSON** y la naturaleza stateless de **REST** son críticos cuando hay 100.000 usuarios queriendo comprar al mismo tiempo en un Cyber Monday?
- 18. Si tuvieras que diseñar el backend para un Homebanking, ¿mantendrías una arquitectura puramente **REST** stateless o considerarías otro enfoque?
- 18.1 Justificá tu respuesta basándote en la sensibilidad de los datos y la necesidad de mantener el estado de la transacción.
- 19 ¿Por qué decimos que un Token **(JWT)** es como el QR que nos permite acceder a la UADE?
- 19.1 Explique qué información viaja en él y por qué no se puede falsificar fácilmente.

Spring Boot

- 1. ¿Qué es un gestor de paquetes?

Ing. Prof. Santos Baltazar Sanchez

- 2. ¿Qué es **Maven** y qué relación tiene con el archive pom.xml en un proyecto Spring Boot?
- 3. ¿Para qué sirve el archivo **pom.xml?**
- 4. ¿Cómo se agregan dependencias a un proyecto Spring Boot?
- 5. ¿Qué ventajas tiene utilizar Spring Boot?
- 5.1 ¿En qué aspectos me ayuda para crear un backend?
- 6. ¿Qué dependencias utilizamos para minimizar el boilerplate en el backend?
- 6.1 Describa cada una y qué viene a solucionar.
- 7. ¿Dónde se configura Spring Boot para conectarse a una base de datos?
- 8. ¿Qué es un servidor web y qué diferencia tiene con **Tomcat?**
- 9. ¿Qué impacto tendría en el tiempo de desarrollo y en la legibilidad del código el NO utilizar dependencias como **Lombok** y **JPA?**
- 9.1 Identificá tres secciones del código donde el boilerplate aumentaría significativamente.
- 10. Compará el uso de **Maven** frente a la descarga manual de archivos .jar.
- 10.1 Porqué para un proyecto profesional de gran escala es indispensable el uso de un gestor de dependencias.

Capas

- 1. ¿Por qué se separan en capas las aplicaciones? ¿Cuál es el propósito?
- 2. ¿Cuáles son las responsabilidades de las capas **Controller, Service, Repository** y **Model?**
- 3. ¿Por qué el **Controller** no debe tener lógica ni manejar entidades?
- 4. ¿Cómo es que se exponen los endpoints del **Controller?** ¿Qué programa los expone?
- 5. ¿Cómo se invoca un endpoint?
- 5.1 ¿Cuál es el formato de la URL?
- 6. ¿Cuál es la responsabilidad del **Service?** ¿Qué es la lógica de negocio?
- 7. ¿Por qué colocar la lógica de negocio en un **Service** y no en un **Controller?** ¿Qué ventajas tiene?
- 8. ¿Qué función cumple la anotación **@Transactional** en el **Service?** Dé un ejemplo.
- 9. ¿Dentro de la capa de Servicio se trabaja con entidades? ¿Por qué?
- 10. En el alta de un producto, ¿dónde pondría el control del campo stock para que no sea negativo? ¿Por qué?
- 11. ¿Qué entiende por **DTO?** ¿Por qué se utilizan cuando reciben y envían datos el **Controller** y el **Service?**
- 12. ¿Entre qué capas deben enviarse **DTOs** y por qué?
- 13. ¿Qué responsabilidad tiene la capa de **Repository?**
- 14. ¿Por qué el **Repository** en muchos casos no tiene código en su cuerpo?
- 14.1 ¿Quién le provee las funcionalidades del **CRUD** y cómo lo hace?
- 15. ¿Cómo hago si quiero hacer una consulta que sea más específica que el **CRUD** en un **Repository?**

- 16. Qué define el **Model?**
- 17. ¿Cómo se crean las tablas de una base de datos utilizando Spring Boot?
- 17.1 ¿Qué anotaciones hay que utilizar?
- 18. ¿Cómo se relacionan diferentes entidades en Spring Boot?
- 18.1 ¿Qué anotaciones son las más importantes y cómo se utilizan?
- 21. ¿Qué sucedería si un desarrollador coloca una consulta compleja de SQL directamente en el **Controller** utilizando **Entitys** en lugar del Servicio con **Repository?.**
  - Analice el impacto en el mantenimiento y en la testabilidad del sistema.
- 22. Si el proceso de Pago falla después de haber descontado el stock, ¿cómo garantiza Spring que los datos no queden inconsistentes en el **Service?**
- 23. ¿Cuál es la razón principal por la que un **Service** devuelve un **DTO** al **Controller** en lugar de la **Entity** directamente?
  - Piense en la privacidad de los datos.
- 24. Un colega sugiere que para ir más rápido deberían usar una sola capa donde el **Controller** llame directamente al **Repository** saltándose el **Service.**
  - Evalúe los riesgos de esta decisión cuando el Marketplace necesite agregar reglas de negocio complejas

Persistencia

- 1. ¿Qué entiende por persistencia en un backend?
- 2. ¿Qué entiende por **ORM?** ¿Qué ventajas nos provee utilizarlo? Dé unos ejemplos.
- 3. ¿Cuál es el **ORM** que utiliza Spring Boot?
- 5. Analice qué sucedería en la base de datos de un Marketplace si borramos una Categoría que todavía tiene Productos asociados.
  - ¿Cómo se comporta **JPA** en este caso si no configuramos el borrado en cascada?
- 6. Si un Producto tiene muchas ImagenProducto, ¿en qué entidad es más eficiente colocar la clave foránea para evitar problemas de rendimiento?

Excepciones

- 1. ¿Qué entiende por una excepción en Java?
- 2. ¿Cómo se manejaban anteriormente las excepciones en Java?
- 3. ¿Cuál es el mecanismo que hoy se utiliza y qué ventajas tiene?
- 4. ¿Qué es una excepción personalizada?
- 5. ¿Qué excepción personalizada crearía si tuviese errores de argumentos inválidos o de recurso no encontrado para las entidades Producto y Marcas?
- 7. Explicar el flujo que sigue una excepción desde que se lanza en el **Service** hasta que llega al **GlobalExceptionHandler.**
  - ¿Cómo logra Spring transformar ese error en un **JSON** legible para el frontend?
- 8. Si estás programando una aplicación legacy, y encontras un código donde el **Controller** tiene bloques try-catch gigantes para manejar errores de base de datos.

    - Que mejorarías según la separación en capas y como lo manejas de manera centralizada
- 9. ¿Qué es más conveniente para un Marketplace validar el stock en el Frontend o en el Backend?
    - Justifique su respuesta desde el punto de vista de la integridad de los datos.

Seguridad

- 1. ¿Qué funciones nos provee Spring Security?
- 2. ¿Qué diferencia hay entre autenticación y autorización?
- 3. ¿Cómo se protegen los endpoints con Spring Security?
- 4. ¿Cómo se indica a Spring Boot cuál es el archivo donde se realizan las configuraciones de seguridad?
- 5. ¿Cuál es el flujo del código fuente para registrar un usuario? Muestre el código de su TPO
    - ¿Cómo sabe qué algoritmo utilizar para encriptar la contraseña antes de guardarla?
- 6. ¿Cómo es el flujo de autenticación o login con **JWT?**
- 7. ¿Qué configuraciones de SecurityConfig utiliza la autenticación para validar un usuario?
- 8. ¿Cómo se genera el token?
    - ¿Qué información necesita para poder generarlo y de dónde se obtiene?
- 9. ¿Dónde se aconseja guardar la llave secreta para generar el token?
- 10. ¿Qué información hay que enviar para llamar a endpoints protegidos con **JWT?**
- 11. ¿Cómo es el flujo en Spring Boot para validar un token? ¿Qué configuraciones de SecurityConfig utiliza?
- 12. En su TPO, cuando se valida el token, ¿consulta la base de datos?
    - ¿Por qué sí o por qué no?
- 13. ¿Cómo se le indica a Spring Security que debe filtrar cada llamado a un endpoint antes de invocar al **Controller?**
- 15. Cuando un usuario envía un **JWT** en el Header, ¿por qué el servidor puede confiar en la información del usuario sin consultar la base de datos en ese instante?
- 16. ¿Por qué es necesario encriptar las contraseñas con **BCrypt** antes de guardarlas?
    - Si un atacante accede a la base de datos, ¿qué información vería y por qué no le serviría?

Despliegue y Containerización:

1. Defina el concepto de **despliegue (deployment)** y explique qué implica "poner en producción" una aplicación en el contexto actual del mercado.

2. Compare el despliegue tradicional en **servidores físicos** frente a las **Máquinas Virtuales (VM)**. ¿Qué limitaciones de escalabilidad y gestión de recursos presentaban estos modelos?
3. ¿Qué es la **containerización**? Explique la diferencia fundamental entre un contenedor y una máquina virtual en términos de uso del kernel y consumo de recursos.
4. Describa las ventajas competitivas de utilizar Docker en el flujo de desarrollo y despliegue de software (portabilidad, aislamiento y repetibilidad).
5. Ante un evento de alta demanda (como un Cyber Monday en Mercado Libre), explique cómo la arquitectura basada en contenedores facilita el **escalamiento horizontal**. ¿Por qué es más eficiente que el modelo tradicional?
6. Defina qué es una **Imagen de Docker**. ¿En qué se diferencia de un contenedor en ejecución?
7. ¿Cuál es el propósito del archivo **Dockerfile**? Mencione al menos tres instrucciones básicas que suele contener (ej. FROM, COPY, RUN).
8. Explique la función de **Docker Hub**
9. ¿Qué es **Docker Compose** y en qué situaciones es fundamental su uso para un desarrollador Full Stack?

Ing. Prof. Santos Baltazar Sanchez